

A Spark batch job processes a $T = 500$ GB dataset stored in $P_0 = 4,000$ HDFS blocks of 128 MB each. The job runs on a cluster of $E = 20$ executors, each with $k = 4$ cores and $m = 16$ GB of memory.

Part A. Parallelism and partitioning.

- Compute the total number of parallel slots S .
- Spark loads one partition per HDFS block by default. How many waves of parallel tasks are required to process all $P_0 = 4,000$ partitions?
- Suppose each task takes $t_{\text{task}} = 6$ s. Compute the total stage time T_{stage} .
- The team is considering coalescing the input down to $P_1 = 80$ partitions before processing. How many waves does this require, and what is the new T_{stage} assuming task time scales linearly with partition size (i.e. each new task takes $t'_{\text{task}} = 6 \cdot (P_0/P_1)$)?
- Comment on whether the coalesce is helpful. Compute the per-task data size for both partition counts and identify any concern.

$$(a) \quad S = E \cdot k = 20 \times 4 = 80 \text{ parallel slots}$$

$$(b) \quad \text{waves} = P/S = 4000 / 80 = 50 \text{ waves}$$

$$(c) \quad T_{\text{stage}} = [P/S] \cdot t_{\text{task}} = 50 \times 6 = 300 \text{ seconds}$$

$$(d) \quad \text{Waves required} = 80 / 80 = 1 \text{ wave}$$

$$t_{\text{task}} = 6 (P_0/P_1) = 6 \times (4000/80) = 300 \text{ seconds}$$

$$\therefore T_{\text{stage}} = 1 \text{ wave} \times t_{\text{task}} = 1 \times 300 = 300 \text{ seconds}$$

(e) Original data per task : $500 \text{ GB} / 4000 = 125 \text{ MB}$

New data per task : $50 \text{ GB} / 80 = 6.25 \text{ GB}$

Coalescing does not speed up processing since both approaches take 300 seconds total.

The concern is memory.

Each execute has 16 GB of memory and 4 cores, meaning each core gets an average of 4 GB. $6.25 \text{ GB} > 4 \text{ GB}$.

$\therefore$ likely causing disk spills or Out-of-Memory (OOM) errors

Part B. Amdahl's Law. The job has a serial driver section (DAG planning, final `collect()`) representing a fraction $1 - f$ of total work; the rest is fully parallelisable.

- (f) For $f = 0.90$ and $S = 80$ slots, compute the speed-up vs. a single slot.
- (g) What is the theoretical maximum speed-up as $S \rightarrow \infty$?
- (h) The team doubles the cluster to $S' = 160$ slots. What is the new speed-up? What relative improvement does this yield compared to $S = 80$?
- (i) For what value of f would a single doubling from $S = 80$ to $S = 160$ yield exactly a $1.5\times$ improvement in speed-up?

$$(f) \text{ speed-up } (S) = \frac{1}{(1-f) + f/S}$$

$$\text{speed-up } (80) = \frac{1}{(1-0.9) + 0.9/80} \approx 8.99 \times$$

$$(g) \text{ The maximum speed-up is } 1/(1-f) = 1/0.1 = 10 \times$$

$$(h) \text{ Speed-up } (s) = \frac{1}{(1-0.9) + 0.9/160} \approx 9.47 \times$$

The relative improvement : $9.47/8.99 \approx 1.053$

$\therefore$ about 5.3% improvement.

$$(i) \text{ speed-up } (160) = 1.5 \cdot \text{Speed-up } (80)$$

$$\frac{1}{(1-f) + f/160} = 1.5 \times \frac{1}{(1-f) + f/80}$$

$$f \approx 0.9938$$

✍ Exercise 2 – Windowing, Watermarks, and Cost Trade-offs

A streaming pipeline ingests IoT sensor readings from $K = 50,000$ distinct devices into Spark Structured Streaming. Each reading is a 120-byte row.

Part A. Window arithmetic. The team computes a rolling average sensor value per device using a sliding window of length $w = 10$ minutes and slide $s = 2$ minutes, with watermark threshold $\theta = 4$ minutes.

- (a) How many overlapping windows does each event belong to?
- (b) How many open windows does Spark maintain at any time?
- (c) Each window keeps one 24-byte aggregate per device (running sum + count). Estimate the total state memory required, in MB.
- (d) The team considers switching to a tumbling 10-minute window. What is the new state memory? By what factor is it smaller?

(a) $K = w/s = 10/2 = 5$ overlapping windows

(b) $N_{open} = (w + \theta)/s = (10 + 4)/2 = 7$ open window

(c) Total memory scales : $N_{open} \cdot K \cdot \text{size} = 7 \times 50000 \times 24 = 8.4 \text{ MB}$

(d) $S = w = 10$ $N_{open} = (10 + 4)/10 = 2$

New memory = $2 \times 50000 \times 24 = 2.4 \text{ MB}$

$8.4 / 2.4 = 3.5 \times$

Part B. Watermarks and late events. The team measures that the event lateness D is approximately exponential with mean $\mathbb{E}[D] = 45$ seconds.

- (e) Compute the rate parameter μ of the exponential distribution.
- (f) With the current $\theta = 4$ min watermark, compute the probability that a given event is dropped.
- (g) The product team requires a drop rate of at most $\alpha = 10^{-4}$ (one in ten thousand). What is the smallest watermark threshold θ^* that meets this target? Express your answer in minutes.
- (h) How much does the new θ^* change the state memory required, compared to $\theta = 4$ min, for the original sliding window? (Assume sliding window with $w = 10$, $s = 2$.)

$$(e) \mu = 1/\mathbb{E}[D] = 1/45 = 0.0222 \text{ s}^{-1}$$

$$(f) P(\text{drop}) = e^{-\mu\theta} = e^{-240/45} = e^{-5.333} \approx 0.0048 = 0.48\%$$
$$\theta = 4 \text{ min} = 240 \text{ s}$$

$$(g) \theta^* = (1/\mu) \cdot \ln(1/\alpha) = 45 \cdot \ln 10^4 \approx 414.46 \text{ seconds}$$

$$\text{In minutes } 414.46/60 \approx 6.91 \text{ minutes.}$$

$$(h) \quad N_{open} = (w + \theta^*) / s = (10 + 6.91) / 2 = 9 \text{ windows}$$

$$\text{New memory} = 9 \times 50000 \times 24 = 10.8 \text{ MB}$$

$$\text{Increase} : 10.8 - 8.4 = 2.4 \text{ MB}$$

Part C. Batch vs. streaming cost model. A batch alternative would re-run the same aggregation every T minutes at cost $C_b = \$8$ per run. The streaming pipeline costs $r_s = \$0.20$ per minute to keep alive.

- (i) At what refresh interval T^* does the streaming option become cheaper than batch?
- (j) The product team requires the result to be no more than 5 minutes stale. Which option is cheaper, and by how much per hour?

$$(i) \quad T \leq C_b / r_s \quad T^* = 8 / 0.2 = 40 \text{ minutes}$$

(j) Staleness is $T = 5 \text{ min}$, since $5 < 40$, streaming is cheaper.

Streaming cost per hour = $60 \times \$0.2 = \12 hr

Batch cost per hour = $(60/5) \times \$8 = \96 hr

$\therefore$ Streaming is cheaper by $\$96 - \$12 = \$84 / \text{hour}$.